

[Pandas](#)[SASS/SCSS](#)[JavaScript](#)[Spring Boot](#)[HTML 5 Reference](#)[Docker](#)

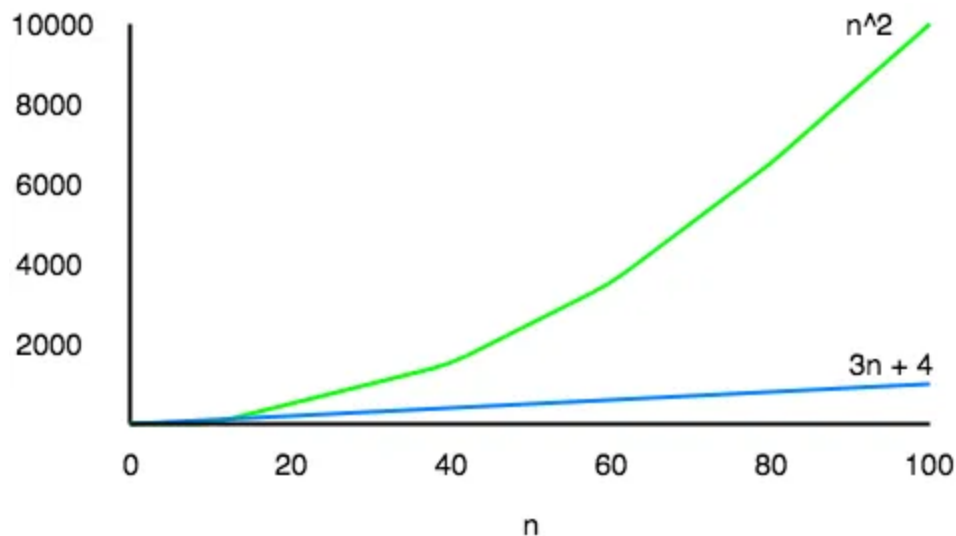
Asymptotic Notations

When it comes to analysing the complexity of any algorithm in terms of time and space, we can never provide an exact number to define the time required and the space required by the algorithm, instead we express it using some standard notations, also known as **Asymptotic Notations**.

When we analyse any algorithm, we generally get a formula to represent the amount of time required for execution or the time required by the computer to run the lines of code of the algorithm, number of memory accesses, number of comparisons, temporary variables occupying memory space etc. This formula often contains unimportant details that don't really tell us anything about the running time.

Let us take an example, if some algorithm has a time complexity of $T(n) = (n^2 + 3n + 4)$, which is a quadratic equation. For large values of n , the $3n$ will become insignificant compared to the n^2 part.

[Index](#)



For $n = 1000$, n^2 will be 1000000 while $3n + 4$ will be 3004 .

Also, When we compare the execution times of two algorithms the constant coefficients of higher order terms are also neglected.

An algorithm that takes a time of $200n^2$ will be faster than some other algorithm that takes n^3 time, for any value of n larger than 200 . Since we're only interested in the asymptotic behavior of the growth of the function, the constant factor can be ignored too.

[Index](#)

What is Asymptotic Behaviour

The word **Asymptotic** means approaching a value or curve arbitrarily closely (i.e., as some sort of limit is taken).

Remember studying about **Limits** in High School, this is the same.

The only difference being, here we do not have to find the value of any expression where n is approaching any finite number or infinity, but in case of Asymptotic notations, we use the same model to ignore the constant factors and insignificant parts of an expression, to devise a better way of representing

complexities of algorithms, in a single coefficient, so that comparison between algorithms can be done easily.

Let's take an example to understand this:

If we have two algorithms with the following expressions representing the time required by them for execution, then:

Expression 1: $(20n^2 + 3n - 4)$

Expression 2: $(n^3 + 100n - 2)$

Now, as per asymptotic notations, we should just worry about how the function will grow as the value of n (input) will grow, and that will entirely depend on n^2 for the Expression 1, and on n^3 for Expression 2. Hence, we can clearly say that the algorithm for which running time is represented by the Expression 2, will grow faster than the other one, simply by analysing the highest power coefficient and ignoring the other constants(20 in $20n^2$) and insignificant parts of the expression($3n - 4$ and $100n - 2$).

The main idea behind casting aside the less important part is to make things **manageable**.

All we need to do is, first analyse the algorithm to find out an expression that defines its time requirements and then analyse how that expression will grow as the input(n) will grow.

[Index](#)

Types of Asymptotic Notations

We use three types of asymptotic notations to represent the growth of any algorithm, as input increases:

1. Big Theta (Θ)

2. Big Oh(O)

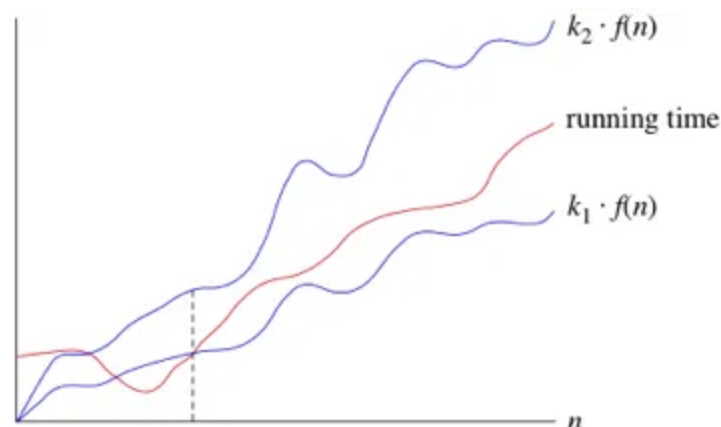
3. Big Omega (Ω)

Tight Bounds: Theta

When we say tight bounds, we mean that the time complexity represented by the Big- Θ notation is like the average value or range within which the actual time of execution of the algorithm will be.

For example, if for some algorithm the time complexity is represented by the expression $3n^2 + 5n$, and we use the Big- Θ notation to represent this, then the time complexity would be $\Theta(n^2)$, ignoring the constant coefficient and removing the insignificant part, which is $5n$.

Here, in the example above, complexity of $\Theta(n^2)$ means, that the average time for any input n will remain in between, $k_1 \cdot n^2$ and $k_2 \cdot n^2$, where k_1, k_2 are two constants, thereby tightly binding the expression representing the growth of the algorithm.



Index

Upper Bounds: Big-O

This notation is known as the upper bound of the algorithm, or a Worst Case of an algorithm.

It tells us that a certain function will never exceed a specified time for any value of input n .

The question is why we need this representation when we already have the big- Θ notation, which represents the tightly bound running time for any algorithm. Let's take a small example to understand this.

Consider Linear Search algorithm, in which we traverse an array elements, one by one to search a given number.

In **Worst case**, starting from the front of the array, we find the element or number we are searching for at the end, which will lead to a time complexity of n , where n represents the number of total elements.

But it can happen, that the element that we are searching for is the first element of the array, in which case the time complexity will be 1 .

Now in this case, saying that the big- Θ or tight bound time complexity for Linear search is $\Theta(n)$, will mean that the time required will always be related to n , as this is the right way to represent the average time complexity, but we use the big- O notation, we mean to say that the time complexity is $O(n)$, which means that the time complexity will never exceed n , defining the upper bound, hence saying that it can be less than or equal to n , which is the correct representation.

[Index](#)

This is the reason, most of the time you will see Big- O notation being used to represent the time complexity of any algorithm, because it makes more sense.

Lower Bounds: Omega

Big Omega notation is used to define the **lower bound** of any algorithm or we can say **the best case** of any algorithm.

This always indicates the minimum time required for any algorithm for all input values, therefore the best case of any algorithm.

In simple words, when we represent a time complexity for any algorithm in the form of big- Ω , we mean that the algorithm will take atleast this much time to complete it's execution. It can definitely take more time than this too.

[< Prev](#)[Next >](#)[Index](#)



DS & Algo MCQ Tests

Crack top Job Interviews.

Explore



Advanced DS

Learn Advanced Data Structures.

Explore



C++ STL Tutorial

C++ Standard Template Library is best suited for Algorithms.

Explore

Popular Links: [normalization in dbms](#) [http in computer networks](#) [deadlock avoidance in os](#) [c programs](#) [page fault in os](#) [paging in os](#) [normalisation in dbms](#) [set operations in dbms](#) [normal forms in dbms](#) [paging in operating system](#) [ktm full form](#) [ng is not recognized as an internal or external command](#) [quality of service in computer networks](#) [paging in os](#) [deadlock prevention in os](#) [icc full form](#) [jcb full form](#) [java 11 features](#) [www in computer network](#) [pipelining in computer architecture](#) [mr full form in medical](#) [star pattern in c](#) [hdlc in computer network](#) [armstrong](#) [number in javascript](#) [pipeline in computer architecture](#) [telnet in computer networks](#) [kfc full form](#) [electronic mail in computer networks](#) [oyo full form](#) [tkinter in python](#) [dig full form](#) [snmp in computer networks](#) [scss tutorial](#) [mr full form in pharmacy](#) [merge sort time complexity](#) [multiplexing in computer network](#) [access modifiers in python](#) [smart ka full form](#) [segmentation in os](#) [leap year program in java](#) [serialization and deserialization in java](#) [thrashing in os](#) [lit full form](#) [lbs full form](#) [process synchronization in os](#) [amul full form](#) [c programming examples](#) [binary search program in python](#) [what is process in os](#) [bcnf in dbms](#) [network model in dbms](#) [banker's algorithm in os](#) [command line arguments in c](#)



Index

studytonight.com



About Us

[Testimonials](#)[Privacy Policy](#)[Terms](#)[Contact Us](#)[Suggest](#)

Part of the **GUVI** universe

 [Learn Coding](#) (for beginners)

 [Tutorial Library](#)

 [Interview Tests](#)

[# Curious](#)

 [Practice Coding](#)

[Index](#)

Coding Courses

[Learn Go Lang](#)[Learn JavaScript](#)[Learn CSS](#)[Learn HTML](#)

Resources

[C Language](#)[C++/STL](#)

[Java](#)[DBMS](#)[Python](#)[PHP](#)[Android](#)[Game Development](#)[Data Structure & Alog.](#)[Operating System](#)[Computer Network](#)[Computer Architecture](#)[Docker](#)[GO Language](#)[GIT Guide](#)[Linux Guide](#)[More...](#)

Interview Tests

[Java Interview Tests](#)[Python Interview Tests](#)[DBMS Interview Tests](#)[Linux Interview Tests](#)[Aptitude Tests](#)[GATE 2022 Tests](#)[More...](#)[Index](#)

Projects/Programs

[Python Projects](#)[C Projects](#)[Python Programs](#)[C Programs](#)[C++ Programs](#)

Java Programs

Index